

3.25 mhashsum ハッシュ法による項目値の合計

hash 法を使って **k=**パラメータで指定した項目を単位にして、**f=**パラメータで指定した項目値を合計する。**msum** との違いは、キー項目の並べ替えが必要ないため、その分処理速度が速い。ただし、キーのサイズ (キー項目のとり値の種類数) が多い場合は処理速度が遅くなる。**msum** と **mhashsum** のどちらを利用するかはデータの内容からユーザーが判断する (後半に示す「ベンチマーク」参照)。

書式

```
mhashsum f= [hs=] [k=] [-n] [i=] [o=] [-assert_diffSize] [-assert_nullkey] [-assert_nulldin] [-assert_nulldout] [-nfn] [-nfno] [-x] [-q] [tmpPath=] [precision=] [--help] [--help1] [--version]
```

パラメータ

- f=** ここで指定された項目 (複数項目指定可) が合計される。
:(コロン) で新項目名を指定可能。例) f=数量:数量合計
- k=** ここで指定された項目 (複数項目指定可) をキーとして集計する。
- hs=** ハッシュサイズ【デフォルト:199999】
処理対象データのキーサイズから、ユーザが消費メモリ量と速度を判断して指定する。指定する値としては素数がよい。
キーサイズが大きいデータに対してハッシュサイズが十分な大ききでなければ処理速度が遅くなる。
ハッシュサイズが十分に大きいと処理速度は速いが、
その分多くのメモリが必要になる (後半に示す「ベンチマーク」参照)。
必要なメモリ量の目安: $K \cdot (24 + F \cdot 16)$ byte, K:キーのサイズ, F:f=で指定した項目数
- n** NULL 値が1つでも含まれていると結果も NULL 値とする。

利用例

例 1: 基本例

「顧客」項目を単位にして、「数量」と「金額」項目の合計を計算する。

```
$ more dat1.csv
顧客, 数量, 金額
A,1,
B,,15
A,2,20
B,3,10
B,1,20
$ mhashsum k=顧客 f=数量, 金額 i=dat1.csv o=rsl1.csv
#END# kghashsum f=数量, 金額 i=dat1.csv k=顧客 o=rsl1.csv
$ more rsl1.csv
顧客, 数量, 金額
A,3,20
B,4,45
```

例 2: NULL 値出力

-n オプションを指定することで、NULL 値が含まれている場合は、結果も NULL 値として出力する。

```
$ mhashsum k=顧客 f=数量, 金額 -n i=dat1.csv o=rsl2.csv
#END# kghashsum -n f=数量, 金額 i=dat1.csv k=顧客 o=rsl2.csv
$ more rsl2.csv
```

顧客, 数量, 金額
A, 3,
B, , 45

アルゴリズムの概要

mhashsum コマンドは分離連鎖法 (separate chaining) によるハッシュ法を用いて実装されている。この方法では、キーを格納する M 個の配列が用意される。キーを構成する文字列をハッシュ関数により 0 から M までの整数 (ハッシュ値) に変換し、配列の番地として利用する。2 つ以上の異なるキーが同一のハッシュ値を持つ (キーが衝突する) 場合は、リンクリストにより格納される。キーを格納すべき番地の探索は定数オーダーであるが、リストの探索は線形オーダーである。そのため、キーの衝突が多発するほど速度は低下する。mhashsum のハッシュサイズはデフォルトで 199999 であり、これはキーのサイズが 20 万までであればリストの平均サイズは 1 以下であることを意味する。実際には、データによっては、キーサイズが小さい場合であってもキーの衝突が多発するケースもあり得る。キーのサイズは `hs=` パラメータで変更できる (上限:1999999)。

ベンチマークテスト

ベンチマークテストの方法

mhashsum コマンド (ハッシュサイズ: 199,999) と msum コマンド (事前に msort コマンドで並べ替えを行う) の計算速度を異なるキーのサイズのデータにおいて計算速度を比較する。キーのサイズとして 100 から 100 万までの 13 種類のデータを対象とした。利用データは、表 3.3 に示されるような key と fld の 2 項目からなる 500 万行の乱数データである。key 項目の値は 6 桁の数値からなる固定長データで fld は 3 桁の数字である。

表 3.3 ベンチマーク用データ (抜粋)

key	value
100020	120
100007	107
100029	129
100065	165
100030	130
100088	188
100055	155
100093	193
100072	172

ベンチマークに利用したコマンド

```
mhashsum による方法
$ time mhashsum k=key f=fld i=dat.csv o=dev/null/

msort+msum による方法
$ time msort i=dat.csv | msum k=key f=fld o=dev/null/
```

実験結果

実験結果を表 3.4 に示す。キーのサイズが小さいとき (~10,000) の速度は、ソーティングする方法に比べて 5 倍程度高速である。キーのサイズが大きくなるに従ってその差は小さくなり、キーのサイズが 80 万を超えるあたりで逆転している。ハッシュサイズが 20 万であることから、リストの平均サイズが 4 の時に逆転していることになる。キーの

値の分布によっても異なるが、この値を mhashsum と msum を使い分ける目安とすればよいであろう。

表 3.4 mhashsum と msum(msort+msum) の速度比較結果

キーサイズ	mhashsum(a)(秒)	msort+msum(b)(秒)	比 (a/b)
100	0.672	2.955	0.227
1,000	0.731	3.981	0.184
10,000	0.814	4.201	0.194
100,000	1.793	4.291	0.418
200,000	2.241	4.336	0.517
300,000	2.604	4.394	0.593
400,000	2.993	4.448	0.673
500,000	3.380	4.497	0.752
600,000	3.793	4.579	0.828
700,000	4.128	4.618	0.894
800,000	4.514	4.667	0.967
900,000	4.901	4.707	1.041
1,000,000	5.352	4.771	1.122

ベンチマーク環境

iMac, Mac OS X 10.5 Leopard, 2.8GHz Intel Core 2 Duo, 4GB メモリ

関連コマンド

msum : 同じ機能をもつコマンドだが、内部的にキー項目の並べ替えを行う。

mhashavg : 同じくハッシュ法を用いた平均計算。